CORAX: Can an AI Agent Audit Quant Research?

1. Background

Quant research is fragile: a one-line error can turn a weak strategy into a persuasive chart. These errors are especially common in financial backtests and AI-generated research code:

- **Lookahead bias**: using future returns or negative shifts to build features;
- **Full-sample normalization leakage**: normalizing across the full sample before the train/test split;
- random splits applied to time-series data;
- **missing transaction costs**: reporting strategy return and Sharpe ratio before costs are deducted;
- **unsupported performance claims**: drawing conclusions with no baseline and no robustness evidence.

This project builds **CORAX**, a Codex-native adversarial audit workflow that treats an AI reviewer as a research auditor: it reads a submitted finance research artifact, emits structured findings, and those findings are compared against human labels. We design an ablation study around CORAX that toggles its components on and off, running four configurations over nine selected stress-test cases to answer one concrete question: **does adding a second agent to an LLM reviewer actually improve audit quality?**

2. CORAX Design

CORAX stands for **Codex-Orchestrated Research Audit eXaminer**, a Codex-orchestrated finance research audit workflow built around adversarial review. Its core design idea is to structurally separate **production, review, and meta-review**, so that the reviewer is not influenced by the producer's language and so that a single audit does not rely on the judgment of a single agent.

The workflow consists of five steps:

1. Producer artifact construction: a benchmark case (submitted research code or a notebook snippet) and a producer claim (the author's self-description of the

research, e.g. "this strategy outperforms buy-and-hold") are assembled into a complete producer artifact that simulates a real research submission under review.

2. Blind brief step: before the material is sent to the reviewer, an independent module (`brief_stripper`) removes all conclusory language and subjective framing, rewriting the producer artifact into a brief that **describes only the code facts and implies no conclusion**. The purpose of this step is to make the reviewer judge the code itself, without being steered by the author's wording.

3. Codex Reviewer: the first LLM agent, which calls a low-cost reviewer model (default `gpt-5.4-mini`) via the Codex CLI. It reads the blind brief (or, in the baseline configuration, the unstripped original artifact) and emits a structured verdict JSON containing: a verdict (PASS/FAIL), an issue list (each issue carries a category, severity, and evidence quote), a confidence score, and counter-arguments (objections the reviewer raises against itself).

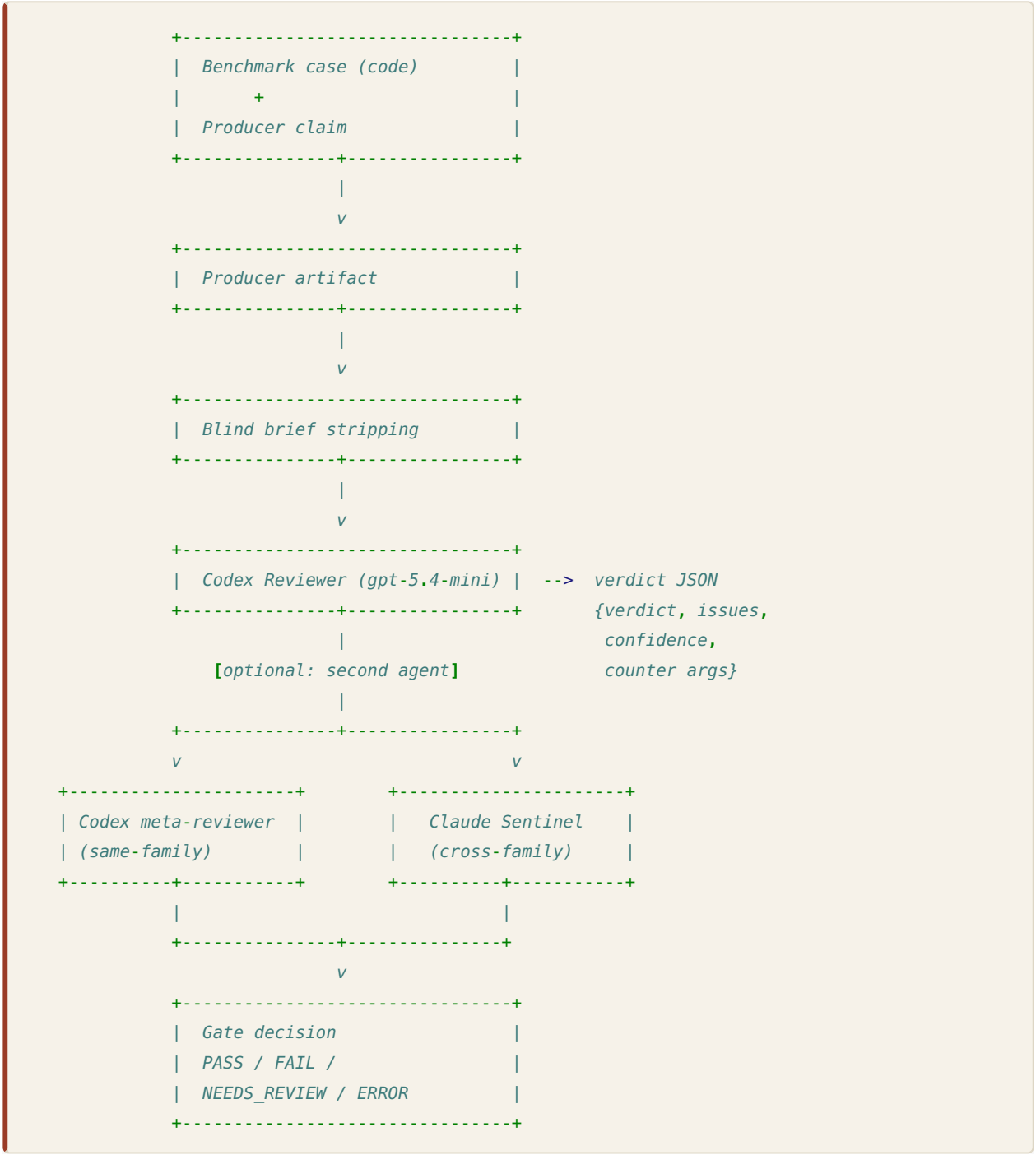
4a. Second agent, Codex meta-reviewer (optional): in the same-family second-agent configuration, the reviewer's output is passed to a **same-family, same-model Codex instance** for meta-review. Using a different prompt, it challenges the first verdict, checking for groupthink, missed concerns, and over-confidence. It evaluates the reviewer's judgment, not the code itself.

4b. Second agent, Claude Sentinel (optional): in the cross-family second-agent configuration, the reviewer's output is passed to an agent from a **different model family** (default Anthropic `claude-haiku-4-5`) for meta-review. It does the same job as the Codex meta-reviewer, checking groupthink, missed concerns, and over-confidence, but because it comes from a different model family, it provides a **cross-family independent perspective**, structurally reducing the risk of a same-family review "agreeing with itself." The name "Sentinel" emphasizes its role as a gatekeeper, rather than a second round of review.

5. Gate decision: based on the combined output of the reviewer and the second agent, the audit is given a final decision that determines the downstream routing of the research artifact:

- **PASS**: the reviewer found no issues and the second agent raised no objection, so the artifact is cleared.
- **FAIL**: the reviewer found issues, or the second agent issued a hard veto, so the artifact must be sent back.
- **NEEDS_REVIEW**: the reviewer passed but the second agent reported high groupthink risk / a missed concern, or the reviewer itself failed, so the case is escalated to a human.
- **ERROR**: the reviewer call failed or returned unparseable output.

Flow Diagram



A Worked Example of the Full Flow

To give the reader an intuitive feel for how CORAX runs, we walk through the full flow with a simplified, fictional example.

Step 0. Input: the original producer artifact

Input code:

```
df['signal'] = df['close'].pct_change().shift(-1)
...
```

Producer claim: "*Our momentum signal generates a remarkable Sharpe of 3.2, clearly outperforming any baseline you'd want to compare against. The methodology is robust and ready for deployment.*"

Step 1. Producer artifact construction

The code and claim above are assembled into a complete producer artifact simulating a research submission. This is what the reviewer sees directly under the most naive configuration (`single_llm`).

Step 2. Blind brief stripping

`brief_stripper` removes all subjective wording and conclusory language from the producer claim, rewriting it into a neutral description. The input the reviewer actually sees:

```
df['signal'] = df['close'].pct_change().shift(-1)
...
```

Neutral description: "*The submission computes a signal from price changes using a one-period negative shift, derives strategy returns from the signal, and reports an annualized Sharpe ratio.*"

Note: the steering phrases `"remarkable Sharpe of 3.2"`, `"outperforming any baseline"`, and `"robust and ready for deployment"` have all been removed.

Step 3. Codex Reviewer output

After reading the blind brief, the reviewer returns a structured verdict JSON:


```
{
  "verdict": "FAIL",
  "issues": [
    {
      "issue": "lookahead",
      "severity": "critical",
      "evidence": "df['signal'] = df['close'].pct_change().shift(-1) uses a negative shift, so
the signal at time t is derived from the return at time t+1."
    }
  ],
  "confidence": 0.97,
  "counter_arguments": [
    "If df['signal'] is intended as a training label rather than a tradeable signal, the
negative shift would be acceptable."
  ]
}
```

Step 4. Second agent output (Claude Sentinel example)

After reading the reviewer's verdict, the Sentinel returns a meta-review JSON following its schema:


```
{
  "groupthink_risk": "LOW",
  "missed_concerns": [
    {
      "severity": "minor",
      "category": "methodology",
      "issue":
"Reviewer flagged lookahead but did not comment on whether transaction costs are modeled
downstream."
    }
  ],
  "verdict_override": "NONE",
  "reasoning": "Reviewer's lookahead finding is well-evidenced and directly tied to the
shift(-1) construction. The counter-argument is acknowledged but does not apply here since the
signal is multiplied by realized returns. No groupthink risk detected."
}
```


Step 5. Gate decision

The reviewer returns FAIL with clear evidence, and the Sentinel's `verdict_override = NONE` means it does not overturn → final **gate decision = FAIL**. The research artifact is sent back.

---

3. Benchmark

CORAX is evaluated on a set of human-labeled finance audit cases. Each case consists of three parts: a **submitted research artifact** (a quant research code or notebook snippet), an independent **annotation** (which issues the code is expected to contain, plus rationale), and the corresponding **real data or workflow fixture**. The benchmark stores the "submitted code" and the "annotation" separately (in `cases.json` and `annotations.json` respectively), ensuring the reviewer cannot see the answers during review.

This ablation study uses a selected set of **nine cases**. These nine were deliberately designed as a stress-test set that covers all the major issue categories, and intentionally includes several **clean near-miss controls** (code that looks suspicious but is actually correct) and one **boundary case whose label is itself ambiguous**, in order to test whether the reviewer is misled by surface-level patterns:

Case	Expected issue	Why included
`btc_future_return_feature`	`lookahead`	obvious future-return leakage
`global_standard_scaler_fit_transform`	`normalization_leakage`	common full-sample scaling bug
`random_split_time_series`	`temporal_split`	random split on time-series data
`cost_variable_declared_not_applied`	`missing_costs`	semantic bug a keyword scanner would miss
`unsupported_claim`	`unsupported_claim`	report-language claim with no evidence
`honest_shifted_momentum`	(clean)	clean control that correctly handles costs
`notebook_transaction_turnover_alignment_ambiguous`	(clean)	boundary case with ambiguous negative-shift use, for failure analysis
`quotemedia_future_winner_signal`	`lookahead`, `missing_costs`	multi-label stock workflow

Case	Expected issue	Why included
`quotemedia_train_window_scaler_clean`	(clean)	clean control with correct scaler usage

The data fixtures for these cases come from real sources: a bundled BTC historical sample, a QuoteMedia stock sample, and two real tutorial notebook workflows; the remaining hand-authored finance snippets are carefully constructed to target specific audit failures.

The benchmark loader validates, at load time, whether fixtures exist, whether data is empty, whether case IDs are duplicated, whether labels are missing, and whether issue types are valid. **If a data file is missing, the loader raises a clear error rather than silently substituting synthetic data**, which guarantees that the evaluation always rests on real data.

---

4. Main Experiment Design: CORAX Ablation

The full CORAX system has two key components: the **blind brief** (stripping producer framing) and the **second agent** (meta-reviewing the reviewer's output). If we only evaluate the full system, we cannot tell how much of the final performance comes from any one of these components. We therefore take an **ablation approach**: toggling these two components on and off, and isolating each component's individual contribution by comparing performance differences across configurations.

The experiment sets up four configurations, run over the same nine cases:

Configuration	Blind brief	Second agent	What it isolates
`single_llm`	No	None	bare baseline: reviewer sees the original artifact with producer framing intact
`blind_only`	Yes	None	isolates the effect of the blind brief alone
`codex_codex`	Yes	Codex meta-reviewer	adds a same-family second agent
`codex_claude`	Yes	Claude Sentinel	adds a cross-family second agent

These four configurations form a cumulative ladder. Between `single_llm` and `blind_only` the only thing that changes is the blind brief, so the performance gap between them is the net contribution of the blind brief; between `blind_only` and `codex_codex` / `codex_claude` the only thing that changes is the second agent, so the gap is the net contribution of the second agent. The comparison between `codex_codex` and `codex_claude` further isolates the effect of whether the second agent comes from the same model family or a different one.

To keep the comparison fair, all configurations use the **same reviewer model, the same cases, the same labels, and the same scoring code**. The only variable is the component being toggled.

On model choice: the main experiment deliberately uses a weak, low-cost reviewer model (`gpt-5.4-mini`), with the cross-family second agent using an equally cheap Claude model (`claude-haiku-4-5`). There are two reasons behind this choice. The first is cost: CORAX is a multi-agent, multi-step workflow in which every case passes through the reviewer and possibly a second agent, so running strong models at scale would quickly make the API cost of a single ablation uncontrollable; only cheap models let the full multi-configuration experiment be run repeatedly within an affordable budget. The second is the experimental design itself: using a weak model turns the experiment into an **architectural stress test**. If the naive reviewer is itself shallow and easily steered by producer framing, then the two architectural components, the blind brief and the second agent, have more room to demonstrate their value. In other words, what we want to measure is not "how accurate some strong model is," but "whether **the audit architecture itself** can squeeze better audit quality out of a not-particularly-reliable reviewer."

---

5. Experimental Results

Using `gpt-5.4-mini` as the reviewer and `claude-haiku-4-5` as the Claude Sentinel, we ran all four configurations over the nine selected cases. The main metrics:

Configuration	Blind brief	Second agent	Precision	Recall	F1	TP	FP	FN
`single_llm`	No	None	0.4615	0.8571	0.6000	6	7	1
`blind_only`	Yes	None	0.8571	0.8571	0.8571	6	1	1
`codex_codex`	Yes	Codex	0.8571	0.8571	0.8571	6	1	1
`codex_claude`	Yes	Claude	0.8571	0.8571	0.8571	6	1	1

Finding 1: The F1 gain comes almost entirely from the blind brief

Comparing layer by layer along the ablation ladder, the first (and only) component that produces an F1 jump is the blind brief. From `single_llm` to `blind_only`, F1 rises from 0.6000 to 0.8571, a gain of 0.26, and the only thing that changed in that step is the blind brief.

The key is where that gain comes from. Note that recall stays at 0.8571 across all three configurations, never changing: the blind brief does not make the reviewer find or miss any real bug (TP stays at 6, FN stays at 1). The gain comes entirely from precision, and the precision gain comes entirely from a collapse in false positives: FP drops from 7 to 1.

The six eliminated false positives are all of the same kind: `unsupported_claim`. The mechanism is clear: under `single_llm`, the reviewer sees both the code and the producer's self-promotional copy ("remarkable Sharpe," "outperforming any baseline"), and so repeatedly triggers the `unsupported_claim` false positive of "this claim has no baseline support." Once the blind brief strips out that steering language, the reviewer returns to judging the code alone, and these framing-induced false positives disappear. In other words, the way the blind brief works is not by making the reviewer smarter, but by **removing the input that would lead it astray**.

Finding 2: The second agent does not change F1, but it changes gate behavior

Continuing up from `blind_only` to add a second agent, whether the same-family `codex_codex` or the cross-family `codex_claude`, the average F1, precision, and recall do not budge at all, holding at 0.8571. Looking at accuracy metrics alone, the second agent appears to contribute nothing.

But accuracy is not the whole story for an audit system. What the second agent really changes is the **gate decision**, the path each case is ultimately routed to:

Configuration	FAIL	NEEDS_REVIEW	PASS
`single_llm`	7	0	2
`blind_only`	7	0	2
`codex_codex`	6	2	1
`codex_claude`	2	7	0

The two configurations without a second agent only ever output a black-and-white PASS / FAIL. Once a second agent is introduced, gate behavior changes

qualitatively: `codex_codex` pushes 2 cases into `NEEDS_REVIEW`, and the cross-family `codex_claude` goes further, marking 7 cases as `NEEDS_REVIEW`, leaving only 2 FAIL and 0 PASS.

This is exactly the expected effect of a cross-family second agent. The same-family `codex_codex`, made of two Codex instances, tends to "agree with itself" (groupthink) and therefore raises fewer objections; whereas the Claude Sentinel in `codex_claude` comes from a different model family, does not share Codex's biases and blind spots, and is more likely to register reservations about the reviewer's judgment, thereby escalating more borderline cases to human review. In a real quant audit pipeline, routing suspicious cases to a human, rather than letting a weak reviewer decide alone, is a more robust form of audit discipline, even when it makes no difference to aggregate F1.

Summary

Taking the two findings together, the ablation yields a clear and layered conclusion: in this CORAX architecture, **what actually improves audit accuracy is the blind brief, while the value of the second agent lies not in accuracy but in audit discipline**. This conclusion is not obtainable by evaluating the full system alone; only by ablating layer by layer can we see what each component is actually doing.

---

6. Case Analysis

Producer Framing Drives Systematic False Positives

The most consistent finding from the ablation is that producer framing in the `single_llm` condition generates false positives for unsupported performance claims across nearly every case. Each case ships with its own self-validating producer claim (for example, `cost_variable_declared_not_applied` asserts the backtest "correctly accounts for transaction costs because a transaction_cost_bps variable is declared"). When the reviewer sees these assertions without a blind brief, it flags the work as making an unsupported claim on top of whatever real issue the case contains, even on cases like `btc_future_return_feature` and `random_split_time_series` that have separate, correctly labeled issues. This produces 7 false positives against 6 true positives and an F1 of 0.60. The blind brief removes those assertions before review and drops false positives to 1, pushing F1 to 0.86. The result is direct evidence that framing control matters more than adding a second-agent layer in this nine-case weak-model run.

QuoteMedia Future Winner Signal: Partial Detection Under All Conditions

`quotemedia_future_winner_signal` is labeled with two issues: `lookahead` and `missing_costs`. All four conditions detect the lookahead bias, since the signal ranks stocks by `future_ret` computed with a negative shift, which is a clear structural error. No condition catches the missing-costs label: the strategy computes `strategy_return = signal * pct_change` but never deducts trading fees. This false negative persists across the blind brief and both dual-agent paths. The case illustrates that the second-agent architecture improves gate conservatism but does not rescue omissions that the primary reviewer misses entirely.

Cost Variable Declared but Not Applied: Semantic Success Case

`cost_variable_declared_not_applied` is the clearest semantic success case in the ablation. The producer framing tells the reviewer that transaction costs are correctly handled because `transaction_cost_bps` is declared, but the submitted code never subtracts that cost from `strategy_return`. All four conditions still detect the expected `missing_costs` issue. This is important because the reviewer is not just matching the word "cost"; it follows whether the cost variable actually affects the return calculation.

Clean Controls Hold, but One Boundary Case Does Not

`honest_shifted_momentum` and `quotemedia_train_window_scaler_clean` are negative-label controls with no planted errors, and both pass cleanly under every condition. This suggests the reviewer is not globally biased toward flagging every clean artifact. The exception is `notebook_transaction_turnover_alignment_ambiguous`: it is labeled clean, but every condition flags it as `lookahead` because it uses a negative shift to align an execution cost to its trade date rather than to build a predictive feature. The blind brief removes most false positives of this kind, but it does not resolve this negative-shift interpretation error, which remains the benchmark's main boundary problem.

---

7. What AI Can Do / What Humans Must Do

In this project, AI acts as a high-throughput audit assistant. In the CORAX workflow, the model reviewer reads submitted backtest code, research writeups, and notebook-derived fragments, and checks for the recurring quantitative research failures the benchmark targets. Its value is not limited to keyword matching. In one benchmark case, the code declares a transaction-cost variable, making the backtest look cost-aware, yet never subtracts that cost from strategy

returns; the AI reviewer caught this data-flow problem, which shows that it can sometimes identify semantic implementation errors rather than only surface-level textual cues. This makes AI well suited to a first round of review: it flags suspicious code patterns so that human reviewers know where to focus their attention.

Human judgment, however, remains essential, because the hard part is not just finding suspicious code but deciding whether it is actually wrong in a financial research setting. Humans need to define the audit criteria: which mistakes count as serious research failures, and which choices may be reasonable depending on timing, execution assumptions, or accounting conventions. The same negative shift, for example, is a serious lookahead problem if it uses future returns as a trading signal, but it may be acceptable if it only aligns transaction costs with the date a trade is executed. Humans also build and validate the benchmark: choosing meaningful failure cases, adding clean near-miss controls, keeping the submitted artifacts separate from the labels, and deciding how ambiguous cases should be scored. Finally, AI outputs need careful human interpretation; a model finding can serve as useful evidence for review, but it is not by itself proof that a strategy is valid or invalid.